

Module 6 — Claude Agent SDK

## Lesson 6.5

# Production patterns

---

*Agents in production need logging, evals, retries, and clear failure modes. The patterns are the same as any other service — applied to a probabilistic backend.*

## Why it matters

A demo agent that "usually works" is fine for show-and-tell. A production agent — one that triages issues, generates content, drives a workflow — needs the same reliability practices as any other service, plus a few specific to LLM behavior. This lesson is the checklist.

## The minimum viable production agent

If you're shipping an agent to production, these are non-negotiable:

- 1 **Structured logging** of every run (input, tools called, output, usage)
- 2 **A timeout / max-turns cap** with a clear failure mode
- 3 **At least one eval** that runs in CI on prompt or tool changes
- 4 **Retry policy** for transient errors (rate limits, network)
- 5 **A way to disable the agent in production without a deploy** (feature flag, env var)
- 6 **Observability dashboards** with cost, latency, success rate

Skipping any of these works until it doesn't, and then it bites hard.

## Structured logging

Log per-run, not per-message. One log line (or JSON object) per agent invocation, including:

- Run ID (uuid)
- Input prompt (or hash if sensitive)
- Tools invoked and counts
- Total turns
- Token usage (input, cached, output)
- Wall-clock time
- Stop reason (`end_turn`, `max_turns`, `error`)
- Final output (or hash, or summary)
- Cost (compute from usage × pricing)

Then per-turn detail in a separate verbose log for debugging.

```
import json, time, uuid

run_id = str(uuid.uuid4())
start = time.time()

result = await run_agent(...)

log.info(json.dumps({
    "run_id": run_id,
    "input_hash": hash(prompt),
    "tools": tool_call_counter,
    "turns": result.turns,
    "usage": result.usage,
    "stop_reason": result.stop_reason,
    "duration_ms": int((time.time() - start) * 1000),
```

```
"cost_usd": compute_cost(result.usage),
}))
```

When something goes wrong, this is what you'll grep.

## Evals

An eval is a test that exercises the agent on known inputs and checks outputs against assertions. Without evals you have no way to know whether a prompt change is an improvement.

Minimum eval setup:

- 1 A small set of representative input prompts (10–100)
- 2 For each, a way to grade the output:
- 3 Exact match (where applicable)
- 4 Regex / structural check (e.g., "output contains a SQL block")
- 5 Cheaper LLM as judge (with a structured rubric)
- 6 Manual review of regressions
- 7 A script that runs all of them and reports pass rate

Run evals:

- On every prompt or tool change (pre-merge)
- On every model version change
- Periodically against production traffic samples

A simple eval prevents the most common production regression: "the new prompt makes the demo case better but breaks three real cases."

## Retries

Three kinds of failure to handle:

| Failure                            | Retry?                                               |
|------------------------------------|------------------------------------------------------|
| Rate limit (429)                   | Yes — exponential backoff                            |
| Transient network error            | Yes — capped retries                                 |
| Model returns gibberish or refuses | Maybe — depends on task; often better to fail loudly |
| Tool error (your code)             | Don't blanket-retry — investigate                    |
| Hit <code>max_turns</code>         | Don't retry — log as failure and alert               |

Wrap the agent call:

```
async def run_with_retry(prompt, max_attempts=3):
    for attempt in range(max_attempts):
        try:
            return await run_agent(prompt)
        except RateLimitError as e:
            wait = (2 ** attempt) + random.random()
```

```
        await asyncio.sleep(wait)
    except (NetworkError,) as e:
        if attempt == max_attempts - 1:
            raise
        await asyncio.sleep(1)
```

Don't retry deterministic failures — those need investigation, not perseverance.

## Feature flags and kill switches

A bug in a deployed agent can do real damage at the rate you call it. Always have a way to disable without a deploy:

```
if os.environ.get("AGENT_ENABLED") != "1":
    log.warning("agent disabled via env var")
    return None
```

Or wire to whatever feature-flag system you already use. On a Friday afternoon when something's misbehaving, you want a one-line fix, not a rollback.

## Cost monitoring

Track per-run cost from `usage`. Alert on:

- Sudden cost spikes (10x baseline)
- Cache hit rate dropping (indicates prompt drift)
- Average turn count rising (indicates the agent is getting confused or hitting `max_turns`)

These are early-warning indicators that the agent is malfunctioning before users notice.

## Model selection

Different tasks suit different models:

| Task                                    | Model  |
|-----------------------------------------|--------|
| Complex reasoning, multi-file refactors | Opus   |
| Most coding tasks                       | Sonnet |
| Narrow mechanical work, classification  | Haiku  |
| Cheap fallback for retries              | Haiku  |

The SDK lets you set `model` per-run. If your agent has multiple steps with different difficulty, consider routing — Haiku for the cheap step, Opus for the reasoning step. (This is more involved than a single `query()` call; you'd compose multiple.)

## Failure modes and what they mean

| Symptom                                   | Likely cause                                           | Fix                                   |
|-------------------------------------------|--------------------------------------------------------|---------------------------------------|
| Agent hits <code>max_turns</code> often   | Task is too complex, or tool failures cause retries    | Decompose task, fix tool errors       |
| Output quality dropped after model update | New model handles your prompt differently              | Re-run evals, tweak prompt            |
| Cost spiked                               | Cache miss, or longer outputs, or new tools added      | Check usage logs, audit cache markers |
| Random refusals                           | Prompt-injected content from user input or tool output | Sanitize, delimit, instruct           |
| Agent does the wrong thing                | Vague tool descriptions, vague system prompt           | Tighten descriptions                  |

## A minimal production wrapper

```

async def production_run(prompt: str) -> RunResult:
    run_id = str(uuid.uuid4())
    if not is_enabled():
        return RunResult.disabled()

    try:
        with timeout(seconds=300):
            result = await run_with_retry(prompt, max_attempts=3)
    except Exception as e:
        log_failure(run_id, e)
        alert_oncall(e)
        raise

    log_success(run_id, result)
    record_metrics(result)
    return result

```

Three concerns covered: enablement, retries, observability. Build the agent inside this wrapper, not as a bare `query()` call.

## Try it

For your example agent:

- 1 Add structured logging of `usage` and `stop_reason` to a JSONL file.
- 2 Add a `max_turns` failure handler that logs loudly when hit.
- 3 Write a minimal eval: 5 hardcoded prompts and a script that runs each, prints success/fail. Iterate the prompt and confirm pass rate.

## Gotchas

- **"It worked in dev" is not a production signal.** Volume changes behavior; rare prompts surface only at scale.
- **Don't tune to a single eval.** You'll overfit. Diversify your eval set as production usage informs you.
- **Cost monitoring lags.** A bug that costs \$1000/hour gets noticed in the morning, after a night of running. Set alerts on cost rate.
- **Models change.** A new model snapshot can change behavior subtly. Pin versions in production, validate before bumping.

## Further reading

- 6.4 Prompt caching — observability includes cache metrics
- Module 7 Automation — wiring agents into infrastructure
- Anthropic eval guidance: <https://docs.claude.com/en/docs/build-with-claude/develop-tests>

## Next

→ 7.1 Headless mode